

## 9/3/26 C programming no. 10 Task-3

```
#include <stdio.h>

struct student
{
    int id;
    char grade;
    float marks;
};

int main()
{
    struct student s;
    printf("size of structure student = %d bytes",
           sizeof(s));
    return 0;
}
```

manual size calculation.

| Data member | Data type | size (bytes) |
|-------------|-----------|--------------|
|-------------|-----------|--------------|

|    |     |   |
|----|-----|---|
| id | int | 4 |
|----|-----|---|

|       |      |   |
|-------|------|---|
| grade | char | 1 |
|-------|------|---|

|       |       |   |
|-------|-------|---|
| marks | float | 4 |
|-------|-------|---|

Total

9 bytes

Actual size using sizeof  
Size of structure student = 12 bytes

why are they different?

The outputs are different

- Manual size: 9 bytes (4 + 1 + 4)

• Size of result: 12 bytes (on most compilers).

This difference occurs because the compiler adds padding bytes b/w members to align data in memory for faster access.  
memory layout.

| member  | size    | offset |
|---------|---------|--------|
| id      | 4 bytes | 0-3    |
| grade   | 1 "     | 4      |
| padding | 3 "     | 5-7    |
| marks   | 4 "     | 8-11   |

So, the total size become 12 bytes instead of 9 bytes.

Hence, they are different because the compiler inserts padding to satisfy memory alignment requirements, improving performance.

The exact size may vary depending on the compiler and system architecture.